

Testing Plan: Authentication & Security (Salting & Hashing)

AgriCareer Tracker

Proyek: AgriCareer Tracker

Lokasi Skrip Uji: PBL/03_Source_Code/database/auth_hashing.py

Lingkungan Eksekusi: FastAPI TestClient, SQLite (backend/adsdb/app.db)

1. Tujuan Pengujian (Objective)

Pengujian ini bertujuan untuk memvalidasi keamanan endpoint autentikasi pada sistem back-end AgriCareer. Secara khusus, pengujian menargetkan keberhasilan pendaftaran pengguna, keandalan mekanisme otentikasi saat login, serta membedah kebenaran penerapan algoritma **hashing** dan penyisipan **salting** yang diproduksi oleh pustaka **passlib**.

2. Cakupan (Scope)

- **Endpoint Registrasi:** POST /auth/register
- **Endpoint Login:** POST /auth/login
- **Struktur Database:** Tabel `users`, spesifik pada kolom `hashed_password` dan *handling* status `is_verified`.

3. Skenario Pengujian (Test Scenarios)

TC-01: User Registration Endpoint Test

- **Deskripsi:** Memastikan *endpoint* /auth/register dapat menerima *payload* pengguna baru dengan benar dan merekam data pengguna ke *database* secara konsisten.
- **Langkah-langkah:**
 1. Siapkan JSON *payload* berisi parameter pendaftaran valid (username, password, full_name, email, dll).
 2. Eksekusi POST /auth/register menggunakan TestClient.
- **Hasil yang Diharapkan (Expected Result):**
 - Status HTTP merespon dengan 200 OK.
 - Data pengguna tersimpan pada database dan *password* berubah menjadi *hash string*.

TC-02: Salting & Hashing Structure Validation

- **Deskripsi:** Membuktikan bahwa *password* tidak disimpan dalam bentuk *plaintext* (teks terang), serta membedah string hash yang dihasilkan untuk memverifikasi keberadaan komponen *salt* dinamis yang dapat mencegah serangan seperti *Rainbow Table*.
- **Langkah-langkah:**
 1. Ambil data `hashed_password` untuk pengguna uji langsung dari `user_repo` (database asli).
 2. Pecah string tersebut berdasarkan karakter delimiter (`$`).
- **Hasil yang Diharapkan (Expected Result):**
 - String memiliki format standar dari `sha256_crypt`: `$5$rounds=<jumlah_iterasi>$<salt_unik>$<password>`
 - Sistem akan mencatat (*log*) pemecahan tersebut untuk menunjukkan letak nilai algoritma, *rounds*, dan *salt* secara transparan.

TC-03: Login Endpoint Test (Valid Credentials)

- **Deskripsi:** Menguji bahwa pengguna yang sah, dengan kredensial (*password*) yang tepat dan status email yang sudah diverifikasi, dapat masuk ke sistem dan menerima token JWT (Bearer Token).
- **Langkah-langkah:**
 1. Simulasikan pembaruan verifikasi email secara manual (`is_verified = True`) pada DB.
 2. Kirim *payload* valid ke `POST /auth/login`.
- **Hasil yang Diharapkan (Expected Result):**
 - Status HTTP merespon dengan 200 OK.
 - Sistem mengembalikan response JSON yang berisi `access_token` beserta tipe *bearer*.

TC-04: Login Endpoint Test (Invalid Credentials)

- **Deskripsi:** Memastikan bahwa sistem keamanan backend dengan tegas mencegah masuknya kredensial (seperti kombinasi *username/password*) yang tidak valid.
- **Langkah-langkah:**
 1. Kirim *payload* berisi *username* valid dan *password* yang salah ke `POST /auth/login`.
- **Hasil yang Diharapkan (Expected Result):**
 - Status HTTP membalas dengan 401 `Unauthorized`.
 - Sistem tidak membocorkan informasi dan mencegah pengguna masuk.

4. Pelaksanaan dan Pelaporan

- **Alat Eksekusi:** Python logging dan Jupyter Notebook (sebagai penyimpan riwayat eksekusi menetap).
- **Hasil (Test Log):** Seluruh output keberhasilan beserta ekstrak *Bearer Token* dan *Salt* dicetak menggunakan modul standar *logger*, menyimulasikan jalannya *CI/CD Integration Testing* yang profesional.